

TUGAS LOAD BALANCING KOMPUTASI PARALEL Materi: Load Balancing (Static vs Dynamic Scheduling)	Nama: Farhan Kamil Hermansyah NRP: 152024150 Mata Kuliah: IFB 206 - Komputasi Paralel dan Sistem Terdistribusi Dosen Pengampu: Lisa Kristiana PhD
---	--

Github :

<https://github.com/Farmil23/IFB-206-KOMPUTASI-PARALEL-SISTEM-TERDISTRIBUSI/>

I. PENDAHULUAN

Pada tugas kali ini, saya diberi kesempatan untuk mencari studi kasus nyata tentang penerapan materi kuliah pertemuan sebelumnya. Materi tersebut berfokus pada penerapan load balancing, sebuah Teknik yang memungkinkan untuk membagi-bagi tugas menjadi beberapa chunk kecil dan dibagikan ke setiap worker.

Sesuai dengan skill yang saya miliki, Saya mengambil studi kasus load balancing pada penerapan AI Agent. Studi kasus ini menggambarkan AI Agent (LLM) yang harus mengretriever informasi di dalam dokumen. Setiap dokumen memiliki beban dan Panjang halaman yang berbeda sehingga setiap proses memiliki waktu yang berbeda. Dengan simulasi ini, saya dapat memahami Teknik Load Balancing secara lebih dalam dan baik.

II. KONSEP TEORI

Berdasarkan materi perkuliahan, terdapat tiga teknik penjadwalan utama:

- **Static Scheduling:** Pembagian tugas dilakukan secara merata di awal eksekusi dengan ukuran potongan (*chunk*) yang tetap, sehingga bobot setiap worker sama rata.
- **Dynamic Scheduling:** Tugas dibagikan saat *runtime*. *Thread* yang selesai lebih cepat akan mengambil tugas berikutnya dari antrian, ini akan membuat worker yang mengganggu karena selesai lebih cepat akan diberikan tugas tambahan.
- **Guided Scheduling:** Ukuran potongan tugas berkurang secara bertahap untuk menyeimbangkan *overhead* dan efisiensi.

III. IMPLEMENTASI KODE (PYTHON)

Saya menggunakan modul multiprocessing **Pool** untuk mensimulasikan kedua teknik tersebut pada skenario **AI Agent** yang memproses dokumen dengan panjang yang berbeda (beban heterogen).

```

from multiprocessing import Pool, cpu_count
import time

def ai_agent_task(doc_info):
    doc_id, duration = doc_info
    print(f" [Worker] Menganalisis Dokumen {doc_id} (Beban: {duration}s)...")
    time.sleep(duration)
    return f"Hasil_{doc_id}"

if __name__ == "__main__":
    # Skenario: 6 Dokumen dengan beban kerja tidak merata (Heterogen)
    # Dokumen 2 sangat berat (4 detik), sisanya ringan (0.5 detik)
    docs = [(1, 0.5), (2, 4.0), (3, 0.5), (4, 0.5), (5, 0.5), (6, 0.5)]
    print(f"--- Simulasi Load Balancing pada {cpu_count()} Cores ---\n")

    # --- TEKNIK 1: STATIC SCHEDULING (Chunksize > 1) ---
    print("1. Menjalankan STATIC SCHEDULING (Chunksize=3)...")
    start_static = time.time()
    with Pool(2) as p_static: # Worker 1 dapat (Dok 1,2,3), Worker 2 dapat (Dok 4,5,6)
        p_static.map(ai_agent_task, docs, chunksize=3)
    time_static = time.time() - start_static
    print(f"Total Waktu Static: {time_static:.2f}s\n")

    # --- TEKNIK 2: DYNAMIC SCHEDULING (Chunksize = 1) ---
    print("2. Menjalankan DYNAMIC SCHEDULING (Chunksize=1)...")
    start_dynamic = time.time()
    with Pool(2) as p_dynamic:
        p_dynamic.map(ai_agent_task, docs, chunksize=1)
    time_dynamic = time.time() - start_dynamic
    print(f"Total Waktu Dynamic: {time_dynamic:.2f}s")

```

IV. HASIL DAN EKSEKUSI

```
D:\LIFE\KULIAH_ITENAS\SEMESTER 4\KOMPUTASI PARALLEL\LECTURE_6_OPENMP_MPI>python MERGE\load_balancing.py
--- Simulasi Load Balancing pada 8 Cores ---

1. Menjalankan STATIC SCHEDULING (Chunksize=3)...
[Worker] Menganalisis Dokumen 1 (Beban: 0.5s)...
[Worker] Menganalisis Dokumen 4 (Beban: 0.5s)...
[Worker] Menganalisis Dokumen 2 (Beban: 4.0s)...
[Worker] Menganalisis Dokumen 5 (Beban: 0.5s)...
[Worker] Menganalisis Dokumen 6 (Beban: 0.5s)...
[Worker] Menganalisis Dokumen 3 (Beban: 0.5s)...
Total Waktu Static: 5.34s

2. Menjalankan DYNAMIC SCHEDULING (Chunksize=1)...
[Worker] Menganalisis Dokumen 1 (Beban: 0.5s)...
[Worker] Menganalisis Dokumen 2 (Beban: 4.0s)...
[Worker] Menganalisis Dokumen 3 (Beban: 0.5s)...
[Worker] Menganalisis Dokumen 4 (Beban: 0.5s)...
[Worker] Menganalisis Dokumen 5 (Beban: 0.5s)...
[Worker] Menganalisis Dokumen 6 (Beban: 0.5s)...
Total Waktu Dynamic: 4.27s
```

Teknik Schedulling	Ukuran Chunk	Total Waktu Eksekusi
Static Schedulling	3	5.34s
Dynamic Schedulling	1	4.27s

V. Analisis dan Kesimpulan

Analisis : Pada kasus kode ini, dapat terlihat perbedaan waktu yang cukup signifikan diantara kedua Teknik scheduling tersebut, pada **Static Schedulling**, terjadi pemrosesan yang lebih lama dikarenakan bobot yang diterima setiap worker sama, namun ada satu masalah, setiap dokumen ada yang bisa dikatakan **memiliki banyak halaman atau kompleks** sehingga membutuhkan waktu lebih lama untuk diproses. Pada contoh ini, dokumen 2 membutuhkan waktu sekitar 4.0s, saya akan simulasikan bahwa beban dokumen 1, 4, 2 itu dilakukan oleh worker 1 dan sisanya yaitu 5, 6, 3 oleh worker 2, Dimana bobot dari worker 1 otomatis tidak sama dengan worker 2 karena pada worker 1 terdapat dokumen yang kompleks. Ini membuat worker 1 memiliki waktu eksekusi yang lebih lama daripada worker 2 karena harus membereskan dokumen 2 yang setidaknya membutuhkan 4 detik daripada dokumen lain yang hanya 0,5 detik saja. Selanjutnya, pada kasus **Dynamic Schedulling** terjadi pemrosesan waktu yang lebih cepat daripada **Static Schedulling**, kita ambil studi kasus yang tadi, hal ini terjadi karena worker 2 berhasil memproses dokumen 5,6,3 lebih cepat dengan total 1,5 detik, sementara worker 1 masih mengerjakan dokumen 2 yang butuh waktu tempo 4 detik belum lagi dokumen 1 dan 4 yang membutuhkan

sekiranya 1 detik untuk diproses keduanya. Pada **Static Scheduling**, Ketika worker 2 sudah menyelesaikan tugasnya, ia tidak akan diberikan tugas tambahan dari worker 1, melainkan dia akan diem dan idle menunggu semua worker selesai, sehingga worker 2 akan menganggur, namun pada **Dynamic Scheduling**, Ketika worker 2 selesai mengerjakan tugasnya, ia tidak akan menunggu worker 1 selesai, melainkan dia akan diberikan tugas tambahan dari worker 1 ini, yang membuat tugas untuk worker 2 akan lebih banyak dibandingkan worker 1, anggap saja Ketika worker 1 memproses dokumen 2, worker 2 selesai Ketika di detik 1,5s lalu dia akan diberikan dokumen tambahan yang seharusnya dikerjakan oleh worker 1, misalkan dokumen 4 diberikan kepada worker 2 dan selesai, dia tidak akan menganggur lagi sebelum worker 1 selesai, dia akan tetap diberikan tugas tambahan terus-menerus jika worker 2 lebih cepat selesai pemrosesan dokumennya.

Kesimpulan: Untuk beban kerja yang tidak merata (seperti *parsing* dokumen AI), teknik **Dynamic** jauh lebih efisien dibandingkan **Static**. Namun, **Static** tetap berguna jika beban kerja antar tugas sudah pasti seragam karena memiliki *overhead* komunikasi yang lebih rendah, dan bisa saja dilakukan penerapan keduanya yaitu Guided/Adaptive scheduling yang setiap saat chunk dikurangi.